The Domino Half Adder

East bay 2: dominoes that add two bits · 20 October 2026

A computer made of nothing but dominoes standing on a table. It adds two one-bit numbers, A and B, and shows the answer as two tiles, CARRY and SUM, which fall over to show a 1:

A	B	CARRY	SUM	
0	0	0	0	0 + 0 = 0
0	1	0	1	0 + 1 = 1
1	0	0	1	1 + 0 = 1
1	1	1	0	1 + 1 = 10 (two)

268 dominoes, laid out over about 1.2 x 1.6 m of table. Open `domino-adder.lua` in bpp, or walk to it in Babbage's Hall. It starts with the tour: all four sums, one after the other.

Keys

Key	
`T`	the tour: 0+0, 0+1, 1+0, 1+1
`A`, `B`	an input: 1 ↔ 0 (the board is set up again)
`G`	GO: push the first domino
`R`	stand them all up again
`Z`	slow motion ↔ real time

How it computes

A falling wave is a 1, no wave a 0. Each input has a **switch**: two brass dominoes, standing (1) or laid aside (0), which leaves a gap no wave can jump. Then three tricks, each tested in the simulation before it was used (see `FINDINGS.md`):

1. **Waves meeting head-on stop each other.** A and B strike the two ends of one line, the collision line; if both come, they meet in the middle and stop.
2. **A branch is taken only by a wave running toward it.** Near each end of the collision line a branch leads up to SUM (blue): the right one is taken only by A's wave (running right), the left one only by B's. So SUM falls when A or B came but not both: **SUM = A XOR B**.
3. **A wave can cut another line.** Its last domino knocks one domino out of that line from the side, leaving a gap. The carry (green) is A's long way round, a delay line; when SUM falls, a branch of it cuts the carry line before A's own wave arrives. So the carry finishes only when A came and SUM didn't: A AND NOT (A XOR B) = **A AND B**.

Each output ends by turning back toward you and knocking over a tell-tale tile: 0 on its front, 1 on its back, which faces up once it has fallen toward you. CARRY is on the left, SUM on the right, so together they read as the binary answer.

The layout is in `domino-layout.lua`, built from the junctions in `domino-lines.lua`. Both are plain Lua, so the same layout drives the simulation and the printed template.

Building it

Run `./make-stls.sh` (it needs OpenSCAD, and bpp to refresh the layout). Everything goes into `stl/`.

Part	How many	
`domino.stl`	268, plus spares	48 × 24 × 7.5 mm. Or use a box of ordinary dominoes of that size.
`tell.stl`	2	the tell-tales: 40 × 60 × 4 mm, digits engraved 0.6 mm deep
`inlay0.stl`, `inlay1.stl`	2 each	the digits, in a contrasting colour, pressed into the engravings
`tile-X-Y.stl`	1 each (39)	the placement template, 200 mm tiles, 1.6 mm thick
`pin.stl`	about 60	3 mm pins joining the tiles at their corners
`sign-*.stl`	1 each	A, B, GO, SUM, CARRY
`template.svg`		the same template 1:1 on paper, if you'd rather not print the tiles

Lay the tiles on a flat, hard table: tile `X-Y` goes X tiles to the right and Y tiles away from you, counting from the corner nearest you on the left. Pin them at their corners. Stand a domino in every pocket. The two pockets of each switch are outlined twice. Leave them empty for an input of 0, and lay those dominoes well aside. Stand the tell-tales in their pockets with the 0 facing you. Then push the domino at GO, gently, away from you.

Print settings: PLA, 0.2 mm layers. Dominoes at 30% infill or more; they should weigh much the same as one another. Tiles flat, solid.

What matters, from the simulation: - **Friction** (domino on domino, and on the table) between 0.3 and 0.5 works. Below that, dominoes slide, and the template's walls are what stop their feet sliding. - **Placing:** the template holds each domino to 0.4 mm and about 2 degrees. All four sums were still right with every domino placed at random within that. - **The cut** in the carry line, where it is spaced wider (4.25 cm apart), is the most sensitive part: the simulation needed between 4.1 and 4.4 cm.

The simulation

Units are centimetres, kilograms and seconds.

- **Dominoes:** boxes of 8 g, with friction 0.4 (a slider, 0.3 to 0.5) and no bounce.
- **Template:** its pocket walls are fixed boxes. Where pockets crowd together (at the junctions), walls that would stand in another pocket are left out, as on the print. The lattice bars between pockets are not simulated; they are no higher than the walls.
- **Steps:** 2400 physics steps a second.
- **Edges:** the printed dominoes' edges are rounded 0.8 mm; the simulation's are square.

Headless testing: `bpp -f domino-adder.lua -n 1750` runs the tour and prints `RESULT domino-adder tour: PASS` or `FAIL`. `DA_IN=AB DA_TOUR=0` runs one sum, and `DA_DEBUG=1` prints which lines fell. Between runs, once nothing is moving, it rests (stops stepping) until the next push; `DA_REST=0` never rests, `DA_REST_LOG=1` shows when it does.